PRE-FINAL YEAR PROJECT · CARDIOSENSE

Predicting heart disease with machine learning

Three feature-selection methods, eleven classifiers and a web app that shows how each one sees the same patient.

[Team members] · Guide: [Guide name]
[Department], [College] · [Month 2026]



THE PROBLEM

Heart disease is the world's leading cause of death, and the warning signs are already in routine test results.

17.9M

deaths a year from cardiovascular disease

Can a model read 13 standard measurements (blood pressure, cholesterol, an exercise ECG) and say who is likely to have heart disease?

Source: World Health Organization, cardiovascular diseases fact sheet

OBJECTIVES

What the project set out to do

01

Compare three feature-selection methods

ANOVA, Chi-square and Mutual Information each pick which of the 13 measurements matter most.

02

Test eleven classifiers on each

From Naive Bayes and KNN to boosting, voting and stacking ensembles, then keep the best.

03

Measure accuracy honestly

A held-out test set and 5-fold cross-validation, with no test patient ever seen in training.

04

Ship a usable, secure web app

Sign-up with email OTP, predictions, side-by-side comparison, dataset and model insights.

THE DATASET

UCI Cleveland heart disease data

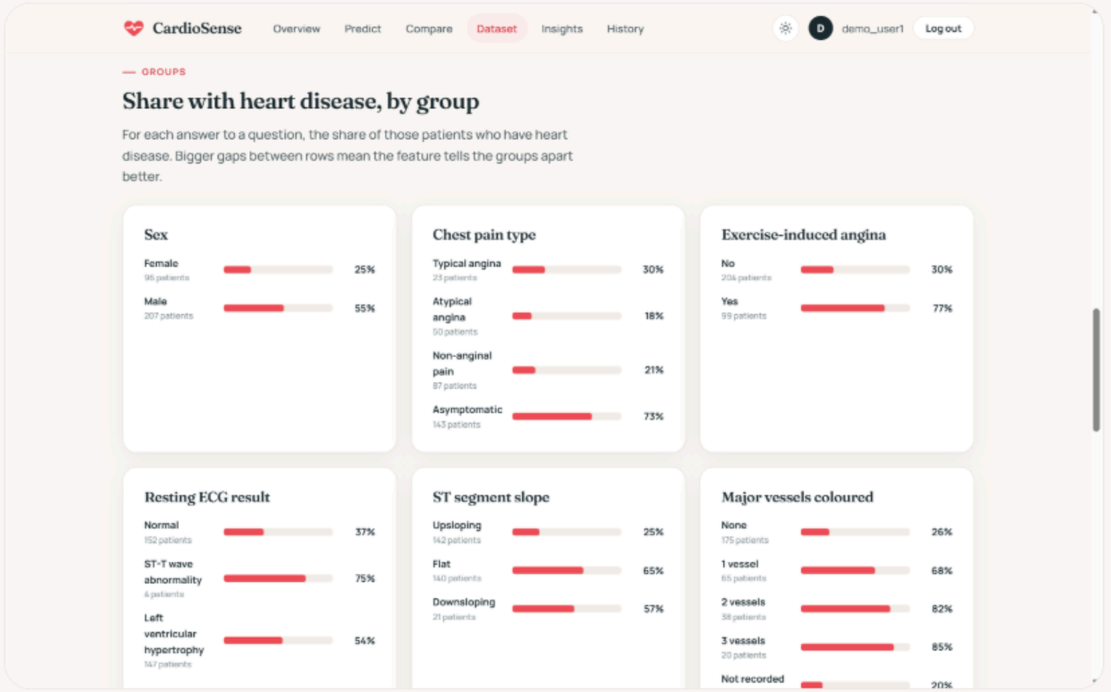
303 patients

13 measurements each

138 with heart disease

Age, sex, chest pain type, blood pressure, cholesterol, fasting sugar, resting ECG, max heart rate, exercise angina, ST depression and slope, vessels on fluoroscopy, and a thallium scan.

Fairly balanced (165 without, 138 with), so a model can't score well by always guessing one answer.



DATA CHECK

The downloaded file had every label reversed

303 / 303

patients matched row by row against the original UCI file. Every “disease” label was really “healthy”, and the other way round.

Clinical sign	“Disease” in the file	Other group
Average age	52.5	56.6
Max heart rate (bpm)	158.5	139.1
Exercise angina	14%	55%
No blocked vessels	79%	33%

The “disease” group looked healthier on every sign. We flipped the labels to match UCI (the original file is kept) and retrained everything.

METHOD · TRAIN.PY

From raw records to a saved model, in seven steps

1
Clean
303 rows, one duplicate dropped: 302

2
Split
61 test patients set aside first

3
Balance
SMOTEENN on the 241 training patients

4
Select
ANOVA k=8, Chi² k=11, MI k=10

5
Train
11 classifiers per method

6
Pick
Best by 5-fold cross-validation

7
Evaluate
Once, on the 61 unseen patients

The rule: nothing that learns from data (resampling, feature selection, model choice) ever sees the test patients. It is repeated inside every cross-validation fold too.

RESAMPLING

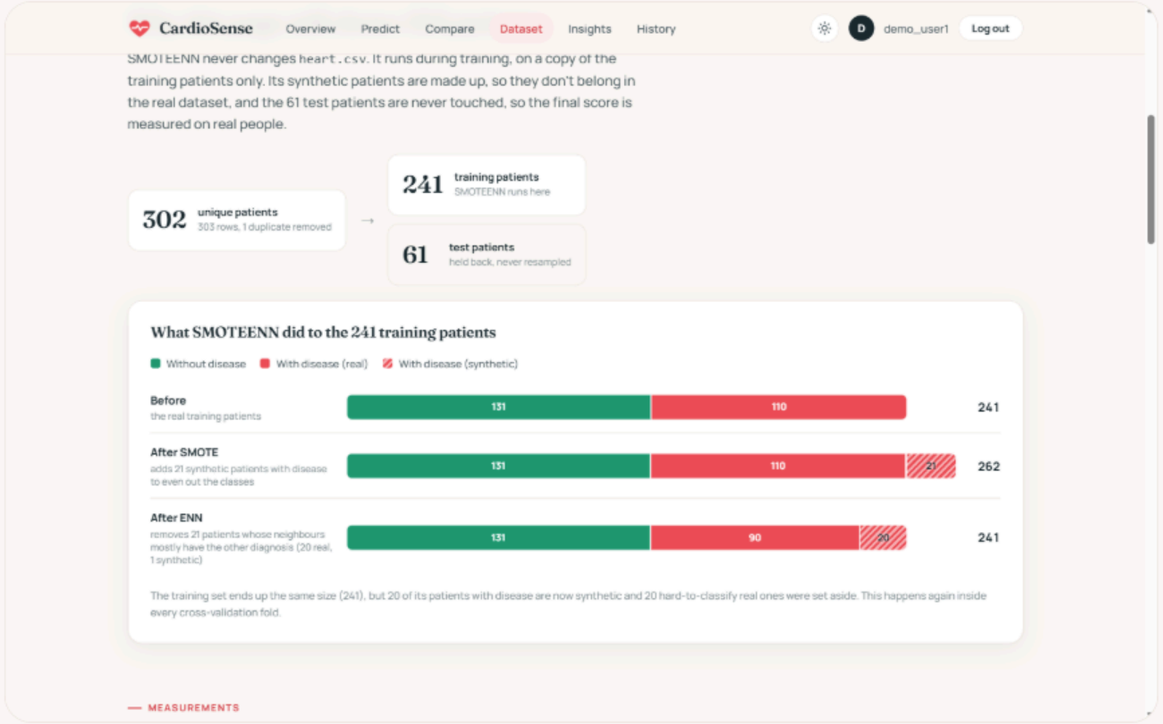
SMOTEENN, tuned for this data

241 → 262 → 241

SMOTE adds 21 synthetic patients with disease to even the classes. **ENN** then removes 21 whose neighbours mostly have the other diagnosis (20 real, 1 synthetic).

The default version threw away three quarters of the data, because raw cholesterol swamped its distance checks. Scaling first, keeping categories as real codes (SMOTENC) and gentler cleaning fixed it.

heart.csv is never changed: synthetic patients exist only inside training.



FEATURE SELECTION

Three ways to decide what matters

ANOVA · keeps 8

Analysis of variance

Keeps features whose averages differ most between sick and healthy patients.

Drops: age, blood pressure, cholesterol, fasting sugar, resting ECG

Chi² · keeps 11

Chi-square test

Keeps features that depend most strongly on the diagnosis.

Drops: fasting sugar, resting ECG

MI · keeps 10

Mutual information

Keeps features that share the most information with the diagnosis.

Drops: age, fasting sugar, resting ECG

All three agree on 8 core signs: sex, chest pain, max heart rate, angina, ST depression, slope, vessels and the thallium scan. No method picked fasting blood sugar.

CLASSIFIERS

Eleven classifiers, one winner per method

Single models

Naive Bayes · KNN · Logistic Regression · SVM · Decision Tree

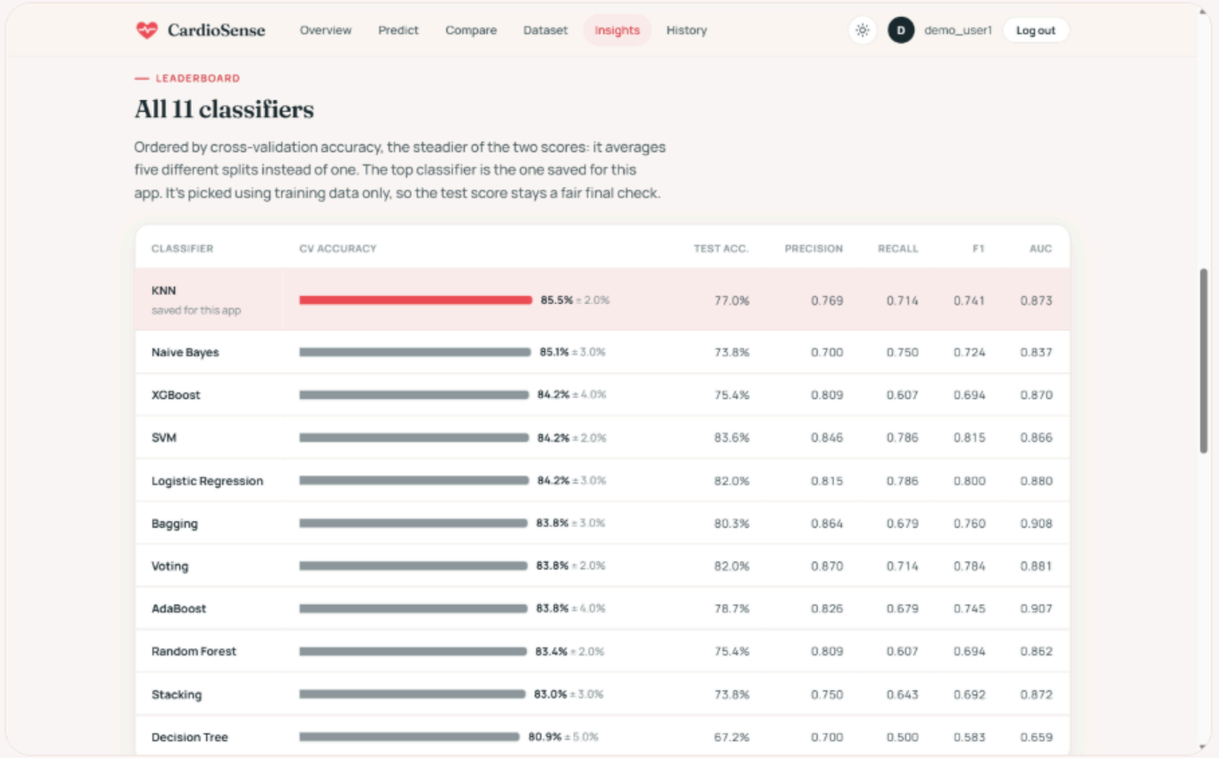
Tree ensembles

Random Forest · AdaBoost · Bagging · XGBoost

Ensembles of ensembles

Soft Voting of all nine · Stacking (boosted stumps + extra trees, combined by LightGBM)

The winner is chosen by **cross-validated accuracy** on training data, so the test set stays a fair final check.



RESULTS

Three methods, effectively tied

ANOVA · KNN

85.5%

cross-validated accuracy, ± 2.3

Chi² · Naive Bayes

84.6%

cross-validated accuracy, ± 2.2

MI · Naive Bayes

83.8%

cross-validated accuracy, ± 2.4

On the 61 unseen test patients	ANOVA	Chi²	MI
Accuracy	77.0%	78.7%	75.4%
Recall (sick patients caught)	71.4%	78.6%	75.0%

Within normal variation: the methods are 1.6 points apart, while one model's score moves about ±2.3 points between folds. Three stronger stacking ensembles, tried on all three methods, scored lower (81.3–83.8%).

HONEST EVALUATION

The first notebooks reported 100%. That number was wrong.

Before

- Resampled before splitting, so near-copies of test patients were in training
- AUC measured on training data
- Always saved Stacking
- Labels reversed

After · train.py

- Test set split off first, never resampled
- AUC and accuracy measured on unseen patients
- Best of 11 chosen by cross-validation
- Labels checked against UCI, row by row

84–86% that you can trust beats a 100% that you can't.

THE APP · HOW IT FITS TOGETHER

One training script, one web app, one source of truth



signup.db

Users (hashed passwords) and each user's prediction history, in SQLite.

metrics.json drives the UI

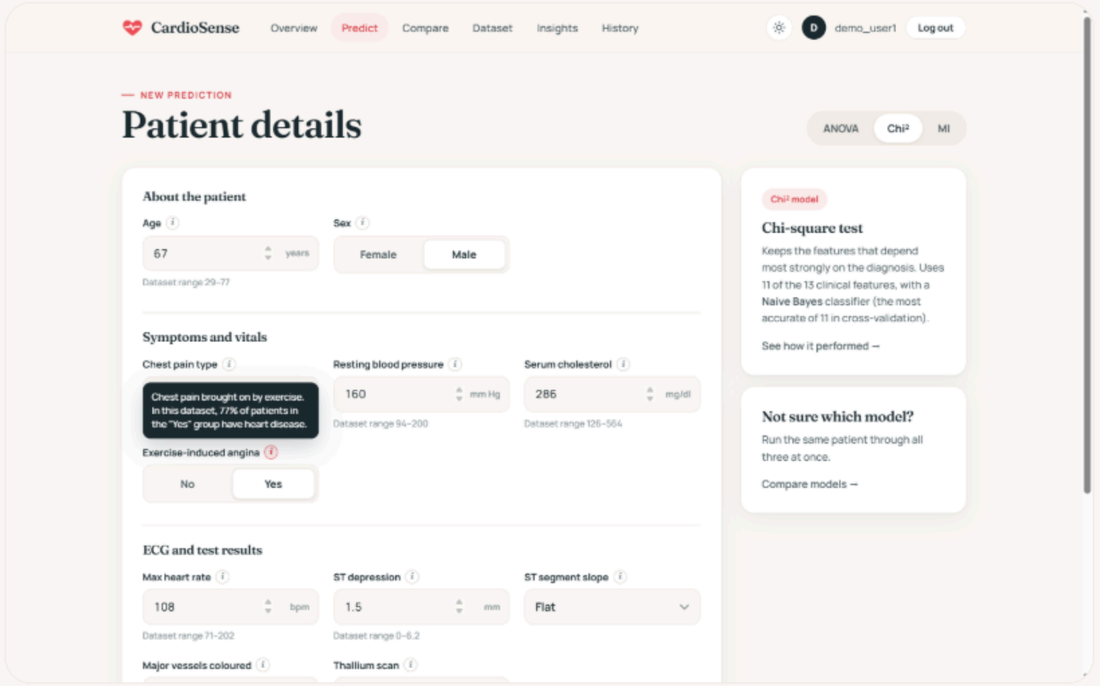
Form fields, leaderboard, ROC curve and confusion matrix all update when you retrain.

Stack

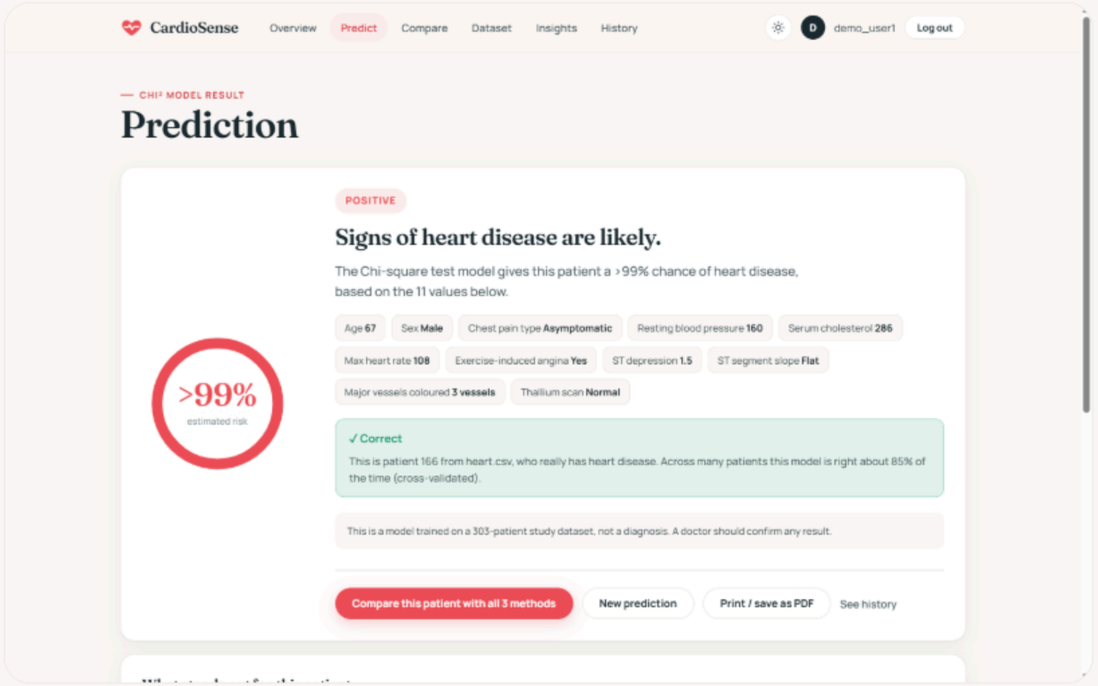
Python 3.13, Flask, scikit-learn, imbalanced-learn, LightGBM; plain HTML, CSS and JavaScript.

APP TOUR · PREDICT

Enter a patient, get a risk score and a real check



The form. Toggles, clinical dropdowns, units and dataset ranges. Every field has an (i) note, and one click fills a real patient from the dataset.



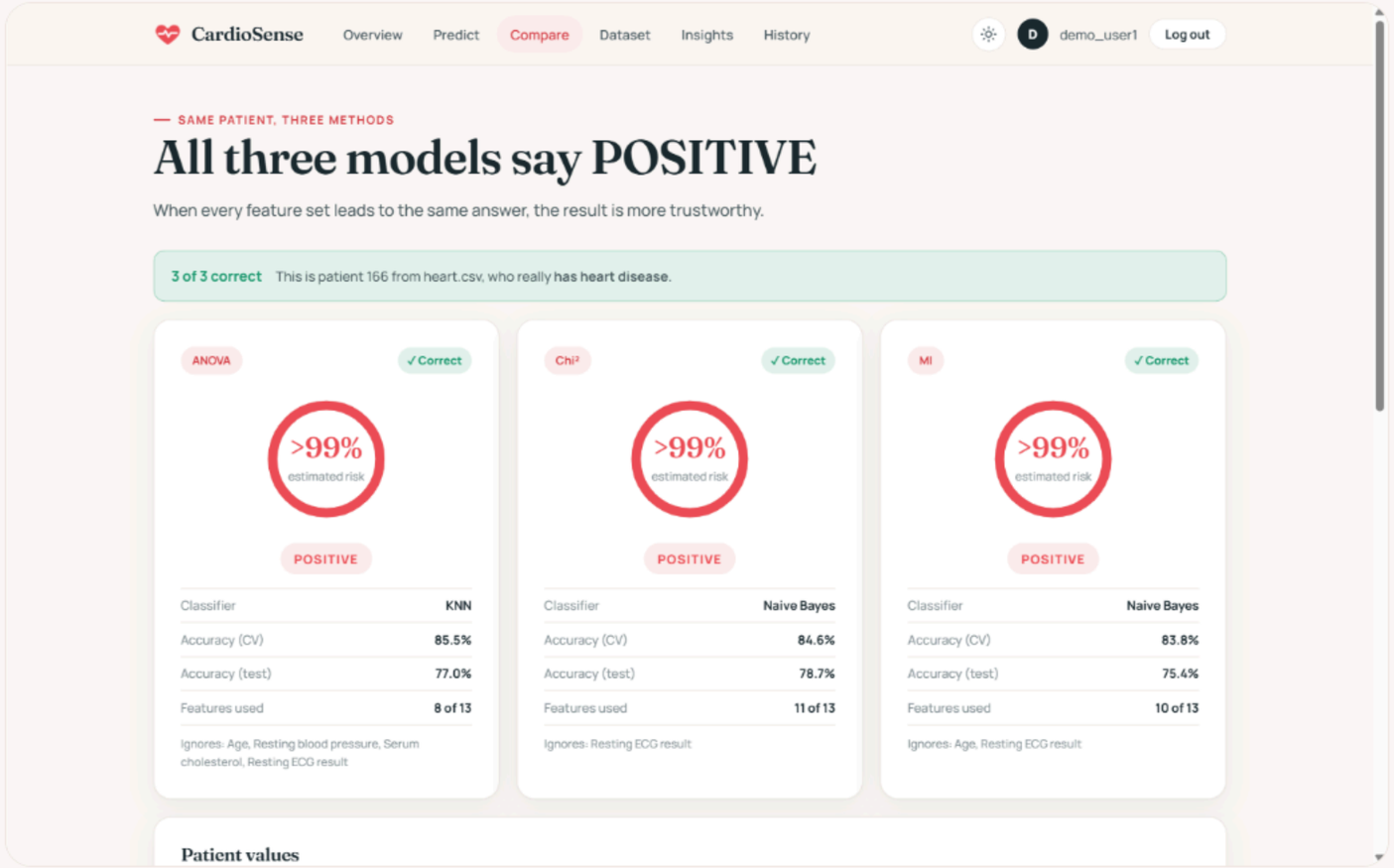
The result. A risk ring and verdict; for a dataset patient, the real diagnosis and whether the model got it right.

APP TOUR · COMPARE

The same patient, through all three methods

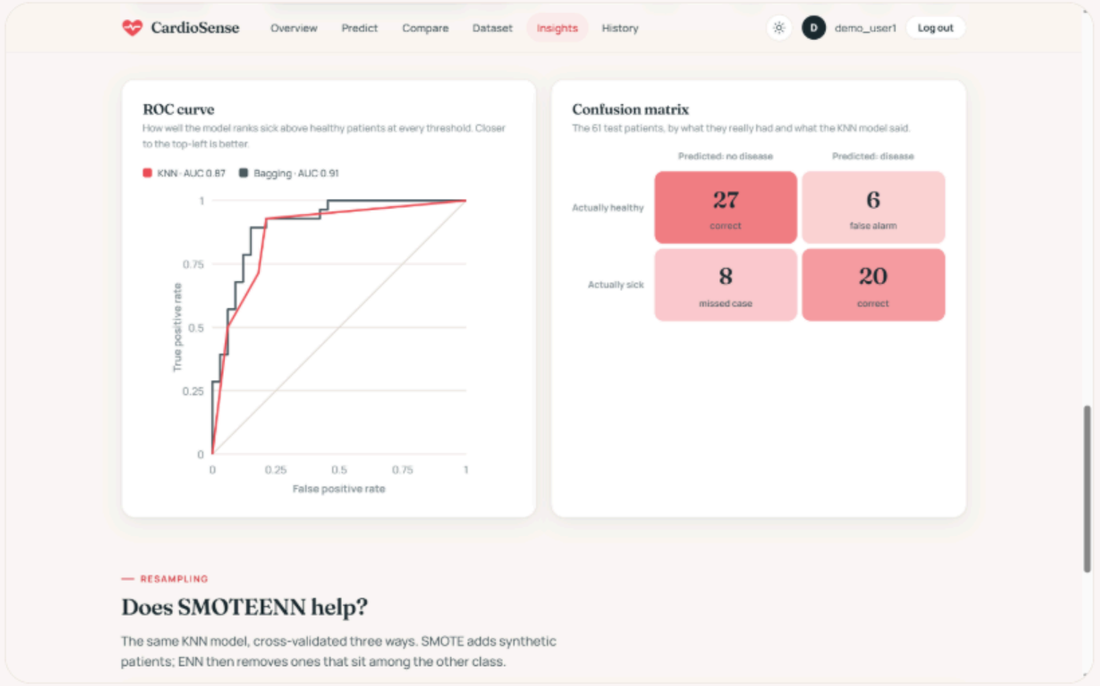
“Compare this patient” on any result carries the same person over. All three models see identical values, so differences come only from the method.

Each card shows the verdict, ✓ **Correct** or ✗ **Wrong** against the real diagnosis, the model’s overall accuracy, and which measurements it ignores.



APP TOUR · INSIGHTS

Every chart drawn live from the training results



ROC curve and confusion matrix for the saved model, on the 61 test patients. Hover a point to read it.



Feature-selection scores, here in dark mode. Also on the page: the 11-classifier leaderboard and the SMOTEENN comparison.

SECURITY AND TESTING

Built to be safe, and checked end to end

Security fixes

- Email password moved out of the code into environment variables
- Passwords stored hashed, never as plain text
- Real login sessions; every page checks them
- 6-digit OTP with a 5-minute expiry and 5 tries
- Login over POST, so passwords never sit in URLs

Testing

45/45
page × screen-size checks

0
console errors

An automated suite (test_app.py) covers sign-up, OTP limits, login, input validation, all 303 patients, the saved models and the compare flow, in about 5 seconds.

LIMITATIONS AND NEXT STEPS

What this project can't claim yet

Limitations

Small data. 302 patients from one hospital; one test patient moves accuracy by 1.6 points.

Overconfident scores. Naive Bayes and KNN often say "<1%" or ">99%", more certain than the data supports.

Not a diagnosis. A screening aid for study, never a replacement for a doctor.

Next steps

Calibrate probabilities so a 70% risk really means 7 in 10.

Validate on new hospitals, handling the missing values in the other UCI sites.

Deploy properly: a production server, HTTPS, and rate limits on OTP emails.

CARDIOSENSE

Thank you.
Questions?

Three methods, eleven classifiers, one honest answer: about 85% from a handful of routine measurements.

[Team members] · [College] · Code: [repository link]

